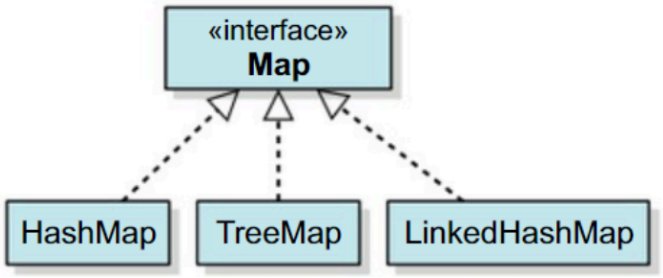
Unidad 11 Interfaz Map

Vistas Collection de los mapas
Implementación de Map

Los mapas o diccionarios son estructuras dinámicas cuyos elementos, que aquí se llaman **entradas** (objetos del tipo **Map.Entry**), son pares clave/valor en vez de valores individuales como en las colecciones. Todas ellas implementan la **interfaz Map**, que no hereda de Collection. Por tanto, los mapas no son colecciones, aunque están íntimamente relacionados con ellas y funcionan dentro del mismo entorno de trabajo. Vamos a usar tres implementaciones de Map:

HashMap
TreeMap
LinkedHashMap

Se diferencian entre sí de forma similar a HashSet, TreeSet y LinkedHashSet.



En un mapa se insertan entradas que constan de una clave, que no se puede repetir, y un valor asociado con ella, que si puede estar repetido.

Las operaciones fundamentales en un mapa son la inserción, la lectura y la eliminación de entradas, aunque veremos algunas más.

Para ilustrar el uso de mapas vamos a empezar utilizando la implementación HashMap, que no garantiza ningún orden de inserción de las entradas, aunque es muy eficiente en cuanto a la velocidad de acceso a los datos. El constructor más sencillo es de la forma:

```
Map<K, V> m = new HashMap<>();
```

donde K es el tipo de las claves y V, el de los valores. Son tipos genéricos que, necesariamente, serán clases o interfaces y no tipos primitivos. Como hicimos con los conjuntos, hemos elegido Map como tipo de la variable m (podríamos haber puesto HashMap), con objeto de garantizar la posibilidad de un mayor

polimorfismo. El comportamiento de m estará determinado por la clase del objeto referenciado.

Ejemplo:

Queremos mantener la información de las estaturas de un grupo de escolares, con entradas en las que figura el nombre del alumno (clase String) como clave y la estatura (clase envoltorio Double) como valor.

```
Map<String, Double> m = new HashMap<>();
```

Ahora podemos usar los siguientes métodos para los mapas.

Insertar entradas

V put(K clave, V valor): si no había ninguna entrada previa con la misma clave, se inserta en el mapa la nueva entrada con esa clave y ese valor, y el método devuelve null. Si ya había una entrada con la misma clave, se sustituye el valor antiguo por el nuevo, sin cambiar la clave, y la función devuelve el valor antiguo

```
m.put("Ana", 1.65);
m.put("Marta", 1.60);
m.put("Luis", 1.73);
m.put("Pedro", 1.69);
```

Con los mapas, igual que con los conjuntos, disponemos también de una implementacion de toString()

```
System.out.println(m);
{Marta=1.6, Ana=1.65, Luis=1.73, Pedro=1.69}
m.put("Pedro", 1.71);
System.out.println(m);
{Marta=1.6, Ana=1.65, Luis=1.73, Pedro=1.71}
```

Eliminar entradas

V remove(Object k): elimina la entrada cuya clave es k, si existe. En este caso, devuelve el valor asociado con esa clave. En caso contrario, devuelve null.

Eliminar todas las entradas de un mapa

void clear(): elimina todas las entradas, dejando el mapa vacío.

Conocer el valor de una entrada a partir de su clave

V get(Object k): devuelve el valor asociado con la clave k o nul1 si no hay ninguna entrada con e s a clave.

```
m.get("Ana");
```

Devuelve 1,65.

Para saber si una determinada clave está presente en un mapa

boolean containsKey(Object k): devuelve true si hay una entrada con la clave .k

Para saber si hay alguna entrada con un valor determinado

boolean containsValue(Object v): devuelve true si hay alguna entrada con valor v.

Dos mapas se pueden comparar entre sí con el método equals(), que devuelve true si ambos tienen exactamente las mismas entradas, independientemente del orden.

Vistas Collection de los mapas

Aunque Map no hereda de Collection, los mapas están íntimamente ligados a las colecciones, de forma que se trabaja simultáneamente con ambas interfaces a través de distintas vistas con estructura de colección.
Por vista de un mapa entendemos una colección respaldada por el mapa original, de forma que cuando accedemos a un elemento de la vista estamos accediendo a la entrada original en el mapa, y los cambios que se hagan en aquella se reflejarán en este.

Hay tres tipos de vistas de un mapa. Vemos los casos siguiendo el ejemplo anterior de las estaturas de un grupo de escolares, con entradas en las que figura el nombre del alumno (clase String) como clave y la estatura (clase envoltorio Double) como valor

Set<K> keySet(): nos devuelve una vista, con estructura Set, de las claves presentes en un mapa.

```
Set<String> claves = m.keySet();
System.out.println(claves);
```

```
[Marta, Ana, Luis, Pedro]
```

con corchetes, ya que es una colección.

Collection<V> values(): devuelve una vista Collection de los valores. Si alguno se encuentra más de una vez en el mapa, también aparece repetido en la coleccion devuelta.

```
Collection<Double> estaturas = m.values();
System.out.println(estaturas);
```

```
[1.6, 1.65, 1.73, 1.71]
```

Set<Map.Entry<K, V>> entrySet(): devuelve una vista conjunto de las entradas, objetos del tipo Map. Entry, de las que se puede obtener la clave con getKey() o el valor con getValue(). Map.Entry es una interfaz (no una clase) implementada en los elementos del mapa y del conjunto devuelto por entrySet().

```
Set<Map.Entry<String, Double>> entradas = m.entrySet();
System.out.println(entradas);
```

```
[Marta=1.6, Ana=1.65, Luis=1.73, Pedro=1.71]
```

Se puede usar la vista de entradas para acceder a las entradas individuales y obtener la clave o el valor, o bien para cambiar este último, con los métodos de la interfaz Map.Entry.

- K getKey(): devuelve la clave de la entrada.
- V getValue(): devuelve el valor de la entrada.
- V setValue (V nuevoValor): asigna nuevoValor a la entrada y devuelve el valor antiguo.

Uno de los inconvenientes de los mapas es que no son iterables. Esto supone que, además de no poder usar iteradores para recorrerlos ni eliminar entradas, tampoco es posible el uso de la estructura for-each, cosa que sí podemos hacer con las vistas, que son colecciones.

Como hemos visto, los cambios que hagamos en ellas se reflejarán en el mapa. En particular, podemos eliminar entradas a través del conjunto de claves devuelto por keySet() con los métodos remove() de Iterator, remove() de Collection, removeAll() o retainAll() . Estos métodos, invocados por la vista, eliminarán las entradas correspondientes en el mapa. Por ejemplo, si eliminamos la clave «Marta» del conjunto claves…

```
claves.remove("Marta");
```

```
{Ana=1.65, Luis=1.73, Pedro=1.71}
```

donde vemos que la entrada correspondiente a Marta ha desaparecido
La única forma segura de eliminar entradas durante un proceso de iteración sobre cual quiera de las tres vistas es el método remove() de la interfaz Iterator. Veamos un ejemplo, pero antes vamos a añadir algunas entradas a nuestro mapa:

```
m.put("Lucas", 1.8);
m.put("Marta", 1.60);
m.put("Jorge", 1.75);
```

```
{Marta=1.6, Ana=1.65, Luis=1.73, Lucas=1.8, Pedro=1.71, Jorge=1.75}
```

Ahora vamos a filtrar el mapa eliminando todos aquellos alumnos con estatura mayor que 1,71. Para ello iteramos sobre el conjunto de las entradas:

```
Set<Map.Entry<String, Double>> entradas = m.entrySet(); /*Set de entradas */
Iterator<Map.Entry<String,Double>> it; //iterador de entradas
for (it = entradas.iterator(); it.hasNext();) {
        Map.Entry<String, Double> e = it.next();
        if (e.getValue() > 1.71) {
                it.remove();
        }
}
```

Obtenemos

```
{Marta=1.6, Ana=1.65, Pedro=1.71}
```

Esto mismo podríamos haberlo hecho iterando sobre la vista de los valores.

```
Collection<Double> estaturas = m.values();
for (Iterator<Double> it = estaturas.iterator(); it.hasNext();) {
        Double v = it.next();
        if (v > 1.71) {
                it.remove();
        }
}
```

En cambio, no podemos añadir entradas a un mapa con add() o addAll() a través de ninguna de sus vistas. La única forma es con el método put() de la interfaz Map

Implementación de Map

La interfaz Map tiene otras dos implementaciones, TreeMap y LinkedHashMap.
TreeMap, a semejanza de TreeSet, tiene una estructura de árbol que permite una inserción ordenada y una búsqueda rápida y eficiente de las entradas. Estas se insertan por orden natural creciente de las claves.

```
TreeMap<String, Double> tm = new TreeMap<>();

tm.put("Ana", 1.65);
tm.put("Marta", 1.60);
tm.put("Luis", 1.73);
tm.put("Pedro", 1.71);

{Ana=1.65, Luis=1.73, Marta=1.6, Pedro=1.71}
```

Podemos hacer que el orden de un TreeMap sea distinto. Para ello le pasamos un comparador al constructor como parámetro de entrada, igual que hacíamos con TreeSet. En cualquier caso, el orden siempre se refiere a las claves, nunca a los valores.
Por último, la implementación LinkedHashMap mantiene el orden en que se van insertando las entradas, de forma similar a lo que ocurre con LinkedHashSet. Es muy eficiente en las operaciones de inserción y eliminación de entradas y algo más lento en las búsquedas.